

E-STORE

Backend Spring Boot

Détail des 5 domaines fonctionnels et de leur code

Module	Full-Stack
Encadrant	Pr. Omar Zahour
Établissement	FSBM — Université Hassan II
Année	2025-2026
Auteurs	Akram Belmoussa & Nouhaila Ben Soumane
Document	2/7 d'une série explicative

Structure générale d'un domaine

Chaque domaine est organisé selon le même schéma, ce qui assure une cohérence forte et une lecture rapide du code.

```
com/estore/&lt;domaine&gt;/
■■■ entity/      Classes JPA (@Entity)
■■■ repository/  Interfaces Spring Data (extends JpaRepository)
■■■ dto/         Objets de transport (sortie/entrée API)
■■■ service/     Logique métier (annotée @Service @Transactional)
■■■ controller/ Endpoints REST (@RestController)
```

1. Domaine customer

Gère les utilisateurs : inscription, connexion, profil. C'est le domaine qui héberge également la sécurité (JWT, BCrypt, filtres Spring Security).

Entité User

```
@Entity
@Table(name = "users", uniqueConstraints = @UniqueConstraint(columnNames = "email"))
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String firstName, lastName;
    @Column(unique = true) private String email;
    @JsonIgnore private String password; // BCrypt
    @Enumerated(EnumType.STRING) private Role role; // USER | ADMIN
    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL) private Profile profile;
}
```

Points clés : @JsonIgnore protège le mot de passe dans toutes les sérialisations, @Enumerated(STRING) stocke le rôle en clair (lisible dans la BD), la relation @OneToOne vers Profile utilise mappedBy = "user" pour éviter les colonnes redondantes.

AuthService — inscription

```
@Transactional
public AuthResponseDto register(RegisterDto dto) {
    if (userRepository.existsByEmail(dto.getEmail())) {
        throw new BusinessException("Cet email est déjà utilisé"); // → 409
    }
    User user = User.builder()
        .firstName(dto.getFirstName()).lastName(dto.getLastName())
        .email(dto.getEmail())
        .password(passwordEncoder.encode(dto.getPassword())) // BCrypt
        .role(Role.USER).build();
    Profile profile = Profile.builder().user(user).build();
    user.setProfile(profile);
    User saved = userRepository.save(user);
    return AuthResponseDto.builder()
        .token(jwtService.generateToken(saved))
        .user(UserDto.from(saved)).build();
}
```

2. Domaine catalog

Catégories et produits. Le ProductRepository expose une recherche paginée combinée (filtre catégorie + mot-clé) via une requête JPQL personnalisée.

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    @Query("""
        SELECT p FROM Product p
        WHERE (:categoryId IS NULL OR p.category.id = :categoryId)
        AND (:q IS NULL OR LOWER(p.name) LIKE LOWER(CONCAT('%', :q, '%'))
            OR LOWER(p.description) LIKE LOWER(CONCAT('%', :q, '%')))
        """)
    Page<Product> search(@Param("categoryId") Long categoryId,
        @Param("q") String q,
        Pageable pageable);
}
```

Avantages : requête unique, paramètres optionnels gérés via IS NULL OR ..., pagination native via Pageable, recherche insensible à la casse.

3. Domaine inventory

Le stock est modélisé par une entité distincte avec relation @OneToOne vers Product. Cela permet de gérer le stock indépendamment du produit (audit, historique, etc.).

```
@Transactional
public void decrement(Long productId, int quantity) {
    Inventory inv = inventoryRepository.findById(productId)
        .orElseThrow(() -> new ResourceNotFoundException("Stock introuvable"));
    if (inv.getQuantity() < quantity) {
        throw new BusinessException("Stock insuffisant pour le produit " + productId);
    }
    inv.setQuantity(inv.getQuantity() - quantity);
    inventoryRepository.save(inv);
}
```

4. Domaine shopping (panier)

Cart possède un @OneToMany vers CartItem avec orphanRemoval=true : la suppression d'un item du panier le supprime aussi en BDD. Le prix unitaire est figé à l'ajout pour garantir la stabilité du panier malgré d'éventuelles évolutions tarifaires.

```
@Transactional
public CartDto addItem(AddToCartDto dto) {
    User user = securityUtils.currentUser();
    Cart cart = getOrCreateCart(user);
    Product product = productService.get(dto.getProductId());

    Optional<CartItem> existing = cart.getItems().stream()
        .filter(ci -> ci.getProduct().getId().equals(product.getId()))
        .findFirst();

    int newQty = existing.map(CartItem::getQuantity).orElse(0) + dto.getQuantity();
    inventoryService.checkAvailability(product.getId(), newQty); // 409 si insuffisant

    if (existing.isPresent()) existing.get().setQuantity(newQty);
    else cart.getItems().add(CartItem.builder()
        .cart(cart).product(product)
        .quantity(dto.getQuantity()).unitPrice(product.getPrice()).build());
}
```

```
    cart.touch();  
    return CartDto.from(cartRepository.save(cart));  
}
```

5. Domaine billing (commande)

Le checkout est l'opération métier la plus critique : elle est transactionnelle (@Transactional) — si l'une des étapes échoue, tout est rollbacké.

```
@Transactional
public OrderDto checkout() {
    User user = securityUtils.currentUser();
    Cart cart = cartService.getOrCreateCart(user);

    if (cart.getItems().isEmpty())
        throw new BusinessException("Votre panier est vide");

    // 1) Vérifier le stock pour TOUS les items AVANT toute modification
    for (CartItem ci : cart.getItems())
        inventoryService.checkAvailability(ci.getProduct().getId(), ci.getQuantity());

    // 2) Créer la commande
    Order order = Order.builder().user(user).status(OrderStatus.PENDING).build();
    BigDecimal total = BigDecimal.ZERO;
    for (CartItem ci : cart.getItems()) {
        order.getItems().add(OrderItem.builder()
            .order(order).product(ci.getProduct())
            .quantity(ci.getQuantity()).unitPrice(ci.getUnitPrice()).build());
        total = total.add(ci.getUnitPrice().multiply(BigDecimal.valueOf(ci.getQuantity())));
        // 3) Décrémenter le stock
        inventoryService.decrement(ci.getProduct().getId(), ci.getQuantity());
    }
    order.setTotalAmount(total);
    order.setStatus(OrderStatus.CONFIRMED);
    Order saved = orderRepository.save(order);

    // 4) Vider le panier
    cartService.clearCart(cart);
    return OrderDto.from(saved);
}
```

Garanties : si le stock est insuffisant en cours de route, ou si la BDD échoue, **aucun** Order n'est créé, **aucun** stock n'est décrémenté, le panier reste intact. C'est l'atomicité ACID en action.